

LocalFinder - Definicion de Backend Ficticio

Especificacion API para un agente backend separado

1. Brief para el agente backend

Construir un backend ficticio consumible por una app Android Kotlin + Compose llamada LocalFinder. El backend debe permitir practicar networking, errores, paginacion, filtros, POST, cache y feature flags. No necesita autenticacion real para el MVP.

- Dominio: busqueda de propiedades en alquiler.
- Ciudad seed recomendada: Curitiba.
- Puede implementarse con JSON Server, Node.js + Express, Ktor Server o Spring Boot.
- Debe devolver JSON estable y facil de mapear a DTOs Kotlin.
- Debe incluir casos de error simulables.

2. Entidades principales

2.1 Property

```
{
  "id": "prop_001",
  "title": "Apartamento amoblado cerca del centro",
  "description": "Apartamento comodo, cerca de transporte publico y supermercados.",
  "city": "Curitiba",
  "neighborhood": "Centro",
  "address": "Rua ficticia, 123",
  "price": 1800,
  "currency": "BRL",
  "type": "APARTMENT",
  "bedrooms": 1,
  "bathrooms": 1,
  "areaM2": 42,
  "isFurnished": true,
  "allowsPets": true,
  "distanceToCenterKm": 1.8,
  "images": [
    "https://example.com/images/property-001-1.jpg",
    "https://example.com/images/property-001-2.jpg"
  ],
  "owner": {
    "id": "owner_001",
    "name": "Marina Souza",
    "rating": 4.8,
    "phone": "+55 41 99999-9999"
  },
  "amenities": ["Wi-Fi", "Garagem", "Elevador", "Portaria"],
  "createdAt": "2026-06-01T10:00:00Z",
  "updatedAt": "2026-06-10T15:30:00Z"
}
```

```
}
```

2.2 UserPreferences

```
{
  "city": "Curitiba",
  "minPrice": 1000,
  "maxPrice": 2500,
  "propertyTypes": ["APARTMENT", "STUDIO"],
  "minBedrooms": 1,
  "allowsPets": true,
  "furnishedOnly": false
}
```

2.3 ContactRequest

```
{
  "id": "contact_001",
  "propertyId": "prop_001",
  "userName": "Josias",
  "email": "josias@example.com",
  "phone": "+55 41 99999-9999",
  "message": "Ola, tenho interesse neste apartamento.",
  "status": "SENT",
  "createdAt": "2026-06-15T12:00:00Z"
}
```

3. Endpoints

3.1 GET /properties

Lista propiedades con filtros, ordenamiento y paginacion.

Query params opcionales:

```
city=Curitiba
minPrice=1000
maxPrice=2500
type=APARTMENT
bedrooms=1
allowsPets=true
furnished=true
sort=price_asc
page=1
limit=20
```

```
{
  "items": [
    {
      "id": "prop_001",
      "title": "Apartamento amoblado cerca del centro",
      "city": "Curitiba",
      "neighborhood": "Centro",
      "price": 1800,
      "currency": "BRL",
      "type": "APARTMENT",
      "bedrooms": 1,
      "bathrooms": 1,

```

```

        "areaM2": 42,
        "isFurnished": true,
        "allowsPets": true,
        "thumbnailUrl": "https://example.com/images/property-001-1.jpg"
    }
],
"page": 1,
"limit": 20,
"total": 52,
"hasNextPage": true
}

```

3.2 GET /properties/{id}

Devuelve detalle completo de una propiedad.

```

{
  "id": "prop_001",
  "title": "Apartamento amoblado cerca del centro",
  "description": "Apartamento comodo, cerca de transporte publico y supermercados.",
  "city": "Curitiba",
  "neighborhood": "Centro",
  "address": "Rua ficticia, 123",
  "price": 1800,
  "currency": "BRL",
  "type": "APARTMENT",
  "bedrooms": 1,
  "bathrooms": 1,
  "areaM2": 42,
  "isFurnished": true,
  "allowsPets": true,
  "distanceToCenterKm": 1.8,
  "images": [
    "https://example.com/images/property-001-1.jpg",
    "https://example.com/images/property-001-2.jpg"
  ],
  "owner": {
    "id": "owner_001",
    "name": "Marina Souza",
    "rating": 4.8,
    "phone": "+55 41 99999-9999"
  },
  "amenities": ["Wi-Fi", "Garagem", "Elevador", "Portaria"],
  "similarProperties": ["prop_002", "prop_003"],
  "createdAt": "2026-06-01T10:00:00Z",
  "updatedAt": "2026-06-10T15:30:00Z"
}

```

3.3 GET /properties/recommended

Devuelve propiedades recomendadas para Home.

Query params:
city=Curitiba

```

{
  "items": [
    {
      "id": "prop_010",
      "title": "Studio moderno no Batel",

```

```

    "city": "Curitiba",
    "neighborhood": "Batel",
    "price": 2200,
    "currency": "BRL",
    "thumbnailUrl": "https://example.com/images/property-010.jpg"
  }
]
}

```

3.4 POST /contact-requests

Crea una solicitud de contacto para una propiedad.

```

Request:
{
  "propertyId": "prop_001",
  "userName": "Josias",
  "email": "josias@example.com",
  "phone": "+55 41 99999-9999",
  "message": "Ola, tenho interesse neste apartamento."
}

```

```

Response success:
{
  "id": "contact_001",
  "propertyId": "prop_001",
  "status": "SENT",
  "createdAt": "2026-06-15T12:00:00Z"
}

```

3.5 GET /contact-requests

Lista solicitudes enviadas.

```

{
  "items": [
    {
      "id": "contact_001",
      "propertyId": "prop_001",
      "propertyTitle": "Apartamento amoblado cerca del centro",
      "status": "SENT",
      "createdAt": "2026-06-15T12:00:00Z"
    }
  ]
}

```

3.6 GET /config

Configuración remota ficticia y feature flags.

```

{
  "minSupportedVersion": "1.0.0",
  "showRecommendedSection": true,
  "enableComparisonFeature": true,
  "enableContactRequests": true,
  "maintenanceMode": false
}

```

4. Codigos de error requeridos

HTTP	code	message	Cuando usarlo
400	INVALID_EMAIL	Email invalido.	Formulario de contacto con email incorrecto.
400	VALIDATION_ERROR	Datos invalidos.	Campos obligatorios ausentes.
404	PROPERTY_NOT_FOUND	Propiedad no encontrada.	ID inexistente en detalle o contacto.
409	PROPERTY_NOT_AVAILABLE	Esta propriedade nao esta mais disponivel.	Propiedad ya no disponible al contactar.
500	SERVER_ERROR	Tuvimos un problema inesperado.	Error interno simulado.

Formato de error:

```
{
  "code": "INVALID_EMAIL",
  "message": "Email invalido."
}
```

5. Reglas de filtros y ordenamiento

- city filtra por coincidencia exacta ignorando mayusculas/minusculas.
- minPrice y maxPrice filtran sobre price.
- type acepta APARTMENT, HOUSE, ROOM, STUDIO.
- bedrooms debe devolver propiedades con dormitorios $\geq$ valor solicitado.
- allowsPets=true devuelve solo propiedades que permiten mascotas.
- furnished=true devuelve solo propiedades amobladas.
- sort acepta price_asc, price_desc, newest y distance_asc.
- page empieza en 1 y limit por defecto puede ser 20.

6. Datos seed recomendados

- Crear minimo 30 propiedades.
- Distribuir barrios de Curitiba: Centro, Batel, Agua Verde, Cabral, Bigorrrilho, Portao, Santa Felicidade, Boqueirao, Cristo Rei, Juveve.
- Incluir variedad de precios entre 900 y 5000 BRL.
- Incluir todos los tipos: APARTMENT, HOUSE, ROOM, STUDIO.
- Incluir propiedades con y sin mascotas, amuebladas y no amuebladas.
- Incluir al menos 5 propietarios distintos.
- Incluir algunos casos para empty state, por ejemplo filtros muy restrictivos.

7. Requisitos no funcionales

- JSON estable y con nombres consistentes.
- Fechas en ISO-8601 UTC.
- IDs string prefijados: prop_001, owner_001, contact_001.
- Latencia simulable opcional entre 300 ms y 1500 ms.
- Errores simulables por query param, por ejemplo ?forceError=500.
- CORS habilitado si se usa desde herramientas externas.
- README con instrucciones para correr localmente.

8. Implementacion rapida sugerida

8.1 JSON Server

```
npm install -g json-server  
json-server --watch db.json --port 3000
```

Ventaja: muy rapido para empezar. Desventaja: filtros complejos y errores personalizados pueden requerir middleware.

8.2 Node.js + Express

Recomendado si el agente backend quiere controlar filtros, validaciones, errores y latencia simulada con mayor libertad.

```
GET    /properties  
GET    /properties/:id  
GET    /properties/recommended  
POST   /contact-requests  
GET    /contact-requests  
GET    /config
```

8.3 Ktor Server

Buena opcion si se quiere practicar Kotlin tambien del lado backend. Mantenerlo simple: rutas, serializacion JSON y datos en memoria.

9. Criterios de aceptacion backend

- GET /properties devuelve lista paginada.
- GET /properties soporta filtros principales.
- GET /properties/{id} devuelve detalle o 404.
- GET /properties/recommended devuelve al menos 5 items.
- POST /contact-requests valida email, propertyId y message.
- GET /contact-requests devuelve solicitudes creadas.
- GET /config devuelve feature flags.
- Errores usan formato { code, message }.
- README explica como correr el backend localmente.